

설비 고장을 예측하는 AI 데이터 분석

Pandas 데이터 선택·필터링

조건 필터링과 정렬

C O N T E N T S

목차

01 조건 필터링 구성 — Boolean Series·복합 조건과 괄호

02 정렬과 종합 구성 — sort_values 정렬·워크플로우 종합

01

조건 필터링과 정렬

Boolean Series로 행 거르기와 복합 조건 묶기

이 단원의 실데이터 — 주조 다이캐스팅

조건 필터링·정렬을 연습할 데이터가 어떤 설비인지 먼저 이해

공정 — 주조 다이캐스팅

용융 금속을 금형에 고압으로 주입해 부품을 성형하는 설비. 한 번의 주입(샷)마다 압력·두께·형체력·품질을 기록한다.

핵심 변수와 물리적 의미

실린더압력·주조압력(주입 압력) · 사이클타임(1샷 소요) · 비스킷두께(잔류 금속 두께) · 형체력(금형 닫는 힘) · 품질등급(양품/주의/불량).

정상과 이상

비스킷두께가 지나치게 두껍거나 사이클타임이 길면 주의 신호, 형체력·압력이 정상 범위를 벗어난 결함 샷은 불량으로 기록된다.

왜 조건 필터링·정렬인가

수백 샷 중 '비스킷두께가 크고 사이클타임이 긴' 위험 샷만 골라(필터) 위험한 순으로 줄 세워(정렬) 우선 점검 대상을 찾기 위해서다.

조건으로 데이터를 뽑는 이유

위치를 몰라도 조건만 알면 원하는 데이터를 다 찾아내는 능력

01 · 위치는 모름

위치를 몰라도 조건만 알면
원하는 데이터를 다 발견

이상 발생 행이 17번인지 342번인지
알 수 없을 때, 조건을 거는 능력이 필요

02 · 현장 출발점

이상 설비 탐지·고장 징후 포착
모두 조건 필터링에서 출발

비스킷두께가 위험하게 높은 설비가 어디야
현장의 진짜 질문에 답하는 도구

위치 기준과 조건 기준

어디에 있는지 알 때와, 어떤 성질인지만 알 때 — 두 갈래로 나뉨

01 · loc · iloc

위치나 이름을 알 때

3번 행처럼
꼭 집어 선택

어디에 있는지 위치 자체를 기준으로 행을 지정

02 · 조건 필터링

위치는 몰라도 성질을 알 때

비스킷두께 높은 행
전부 선택

데이터 내용을 기준으로 행을 찾는다는 점에서 근본적으로 다름

비교 연산 다시 보기

조건을 만들려면 크다·작다·같다 같은 비교 연산

조건을 만들려면 먼저 **비교**를 할 줄 알아야 함

비교의 결과는 언제나 참 또는 거짓 둘 중 하나

비스킷두께 87도는 80보다 큰가에 대해 참, 65도는 거짓

> 크다 < 작다 == 같다 != 다르다

>= 크거나 같다 <= 작거나 같다

파이썬 기초에서 배운 비교 연산자 그대로 사용

비교 연산 주의점

가장 흔한 두 실수 — 등호 개수, 그리고 경계값 포함 여부

01 · 같다는 ==

= 값 넣기,
== 같은지 비교

등호 하나는 값을 넣어라,
등호 두 개가 같은지 비교하라는 뜻

02 · 이상과 초과

13 이상은 80 포함,
80 초과는 80 제외

위험 기준을 정할 때 경계값을 넣을지
말지를 분명히 해야 의도대로 걸러짐

Boolean Series 만들기

열 전체에 조건을 걸면 행마다 참·거짓이 담긴 한 줄짜리 시리즈가 생성

01 · 한 번에 비교

열 모든 값이
한꺼번에 비교

df['비스킷두께'] >= 80 이라고 하면
비스킷두께 열 모든 값이 80과 비교됨

02 · Boolean Series

행마다 참·거짓이 담긴
참·거짓 목록

어느 행이 조건에 맞는지
알려 주는 지도 역할

Boolean Series 코드

열 전체 비교로 참·거짓 목록 만들기

CODE

```
조건 = df['비스킷두께'] >= 13  
print(조건.head()) # True/False  
print(조건.sum()) # 6 (참 개수)
```

sum은 참(True)을 1로 계산 — 참 개수를 셀 수 있음

Boolean Series로 행 추출

앞에서 만든 참·거짓 목록을 데이터프레임 대괄호에 넣어 추출

앞에서 만든 참·거짓 목록을 데이터프레임 대괄호에 넣기

`df[df['비스킷두께'] >= 80]`이면 비스킷두께 13 이상인 행만 추출

안쪽이 참·거짓 목록을 만들고 바깥 대괄호가 참인 행만 추출

STEP 1. 조건이 참인 행은 어디지 하고 참·거짓 목록을 만들기

STEP 2. 그 참인 행만 꺼내라고 데이터프레임 대괄호에 넣기

체로 거르듯 추출

df[조건]은 참인 행만 통과시키는 체와 같음

01 · 필터링 = 거르기

참이면 남기고
거짓이면 버림

대괄호 안에 참·거짓 목록이 들어오면
각 행과 순서대로 맞춰 봄

02 · 결과도 다시 표

골라낸 결과에
통계·추가 선택 가능

골라낸 결과도 다시 표여서
통계나 추가 선택을 이어붙임

실습 1. 단일 조건으로 행 추출하기

목표

조건을 만들고 그 조건으로 원하는 행만 추출

단계

- 비교 연산자로 실린더압력 기준의 조건식을 만들어 Boolean Series 생성
- sum으로 조건을 만족하는 행 개수 확인
- 만든 조건을 데이터프레임 대괄호에 넣어 행 추출

예상 결과

참 개수와 추출 행 수가 같게 출력 (실린더압력 230 이상 19건)

실습 2. 임계값 넘는 설비 골라내기

목표

실제 제조 데이터에서 위험 임계값을 넘는 설비 추출

단계

- 비스킷두께 열에 비교 연산자로 임계값 기준 조건 생성
- 조건을 대괄호에 넣어 임계값 초과 설비만 추출
- 결과에서 샷와 비스킷두께 열만 골라 확인

예상 결과

비스킷두께 16 이상 40건, 샷·비스킷두께 목록 출력

조건 2개 이상, and·or가 안 되는 이유

노베이스가 거의 100% 막히는 지점 — and로 묶으면 오류

01 · and·or의 본래 용도

값 하나끼리
묶는 도구

참 또는 거짓 값 하나끼리 묶을 때
쓰는 파이썬 기본 도구

02 · Boolean Series는 다름

여러 행짜리는
별도 기호 필요

행마다 참거짓이 잔뜩 든 Series를
and는 어떻게 묶을지 몰라 오류

왜 오류가 나고 어떻게 푸는가

모호 오류는 별도 기호 &·| 로 해결

01 · 오류 원인

전체가 참인지 거짓인지
단정 못해 모호 오류

Boolean Series는 참도 거짓도 섞여 있어
전체를 하나로 단정할 수 없음

02 · 해결 신호

and·or 대신
& 와 | 사용

별도 기호를 쓰면
행마다 짝을 맞춰 묶어 줌

& 와 | 그리고 괄호 규칙

각 조건은 반드시 괄호로 감싸기 — 가장 흔한 실수

두 조건 **모두 만족**은 &, **하나라도 만족**은 |

각 조건은 **반드시 괄호**로 감싸기

괄호를 빼면 &가 비교보다 먼저 계산되려 해서 오류

```
(df['비스킷두께']>=80) & (df['사이클타임']>=4.0)
```

```
df['비스킷두께']>=80 & df['사이클타임']>=4.0
```

복합 조건 코드

각 조건을 괄호로 감싸 & 또는 |로 연결

CODE

```
both = df[(df['비스킷두께']>=13) & (df['사이클타임']>=25)]
either = df[(df['비스킷두께']>=13) | (df['사이클타임']>=25)]
print(len(both), len(either)) # 5 7
```

함께 따라 칠 코드는

부정 조건 ~

물결 기호 ~를 조건 앞에 붙이면 참과 거짓이 뒤집힘

물결 기호를 조건 앞에 붙이면 참과 거짓이 뒤집힘

`df[~(df['품질등급'] == '불량')]`는 불량가 아닌 행

복잡한 조건 전체를 통째로 뒤집을 때 특히 편리

파이썬 기초의 **not**과 같은 역할이지만, Boolean Series에는 ~를 사용

키보드에서 보통 숫자 1 왼쪽에 위치

부정 조건 활용

예외를 제외하고 볼 때, 그리고 isin과 결합할 때

01 · 예 외 제 외

정상이 아닌 모든 설비,
특정 품질등급 제외 나머지

예외를 제외하고 볼 때 유용
특정 품질등급을 뺀 나머지를 찾기

02 · isin 결 합

~isin(목록)으로
목록에 없는 행 추출

점검 끝난 설비를 목록으로 두고
아직 안 한 나머지만 추출 때 편리

isin으로 여러 값 매칭

값이 주어진 목록 안에 있는지 한 번에 검사

isin은 **값이 목록 안에 있는지**를 한 번에 검사

`df['품질등급'].isin(['양품', '주의'])`처럼 리스트로 전달

세로 막대를 여러 번 쓰는 것보다 훨씬 깔끔

양품거나 주의거나 설비c인 행을 찾을 때 — 세로 막대로 세 번 잇는 대신 isin으로
품질등급이 그중 하나인 행에 참이 되는 Boolean Series 생성

isin 이해와 활용

파이썬 in의 데이터판 — 분류가 있는 데이터에서 특히 유용

01 · in의 데이터판

**x in 리스트를
열 전체에 한꺼번에**

파이썬의 포함 여부 검사를
열 전체에 적용 — 괄호 안에 리스트 필수

02 · 분류 활용

**품질등급 여러 종류 중
몇 개만 골라낼 때**

품질등급이 L, M, H 세 종류일 때
L 또는 H만 골라내는 식 — 값이 많을수록 장점

between으로 범위 조건

값이 두 경계 사이에 있는지 한 번에 검사 — 양쪽 경계 모두 포함

01 · 범위 검사

```
df['비스킷두께'].between(70, 80)
```

한 줄로 범위 검사

같은 열에 두 경계가 있을 때
복합 조건보다 짧고 읽기 쉬움

02 · 양 끝 포함

`between(70, 80)`은
70과 80도 포함

70 초과 80 미만으로 착각하면
경계값이 빠져 결과가 미묘하게 달라짐

between 활용

복합 조건 대체, 이상치 결합으로 활용 폭이 넓음

01 · 복합조건 대체

같은 열 범위는
짧고 읽기 쉬움

비스킷두께가 두 번 반복되지 않고
괄호 한 쌍으로 충분

02 · 이상치 결합

~between으로
정상 범위 밖 이상치

between 앞에 물결을 붙이면
정상 범위 밖 측정 골라냄

문자열 조건 `str.contains`

글자 데이터에서 특정 단어 포함 여부를 검사

`str.contains`는 글자 데이터에서 **특정 단어 포함 여부**를 검사

`df['샷'].str.contains('L')`처럼 작성

숫자 열이 아니라 **글자 열에만 사용 가능**

열 이름 뒤에 `.str.contains`를 쓰고 괄호에 찾을 글자를 따옴표로 전달

여기서 `str`은 이 열을 글자로 다루겠다는 신호

같다와 포함의 차이

정확히 일치냐, 일부라도 포함이냐 — str.contains의 핵심

01 · 같다 ==

값이 정확히 일치
'양'은 정확히 양일 때만

품질등급이 양품처럼
'양품'은 ==양으로는 못 찾음

02 · 포함 contains

일부라도 포함하면 참
양품도 '양' 포함

'양'이 들어간 모든 품질등급을
한 번에 발견

조건 도구 한눈에 정리

자유롭게 조합하면 아무리 복잡한 조건도 표현 가능

도구	용도	예시
&	모두 만족	(A) & (B)
	하나라도 만족	(A) (B)
isin	목록 매칭	.isin(목록)
between	범위 검사	.between(소, 대)
~	부정·뒤집기	~(조건) · ~isin · ~between

실습 3. 두 조건 묶기

목표

두 조건을 그리고·또는로 묶어 행을 추출

단계

- 비스킷두께 조건과 사이클타임 조건을 각각 괄호로 감싸기
- 두 조건을 그리고 기호로 묶어 모두 만족하는 행 추출
- 같은 두 조건을 또는 기호로 묶어 결과 수 비교

예상 결과

그리고는 12건, 또는는 94건으로 개수 차이 확인

실습 4. 부정·목록·범위 조건

목표

부정·목록 매칭·범위 조건을 각각 적용

단계

- 물결 기호로 고장이 아닌 설비만 뒤집어 추출
- isin으로 품질등급이 특정 목록에 속하는 행 추출
- between으로 실린더압력가 지정 범위에 든 행 추출

예상 결과

순서대로 192건·94건·108건 출력

조건 필터링 패턴 정리

모든 조건 필터링의 기본 패턴은 `df[조건]`

모든 출발점은 **Boolean Series**

바깥 구조는 언제나 `df[ ]` — 안의 조건만 바꾸면 됨

단일 조건이든 복합 조건이든, 결국은 이 패턴 안에 어떤 조건을 넣느냐의 문제

`df[ 어떤 조건이든 ]`

바깥 구조는 고정, 안의 조건만 바뀜

조건 만들기과 묶기

두 단계로 정리 — 만들기과 묶기

01 · 조건 만들기

비교로 Boolean Series 생성
df[조건]으로 추출

크다·작다·같다로 비교해
참·거짓 시리즈를 만들기

02 · 조건 묶기

& · | · ~
isin · between · contains

이 도구들을 조합하면
어떤 조건도 표현 가능

가장 흔한 두 가지 실수

이번 파트에서 가장 자주 나오는 실수 두 가지

01 · and·or 사용

복합 조건에 **and·or**
쓰면 오류

반드시 **&** 와 **|**
별도 기호를 사용

02 · 괄호 누락

각 조건 괄호로
안 감싸면 오류

(조건1) & (조건2)
형태를 몸에 익히기

02

정렬과 종합

sort_values로 줄 세우고 선택→필터→정렬 워크플로우 종합

정렬이 필요한 이유

데이터를 위험한 순서로 줄 세우면 우선순위가 드러남

01 · 우선순위

위험한 순서로 줄 세우면
우선순위가 드러남

비스킷두께 큰 순으로 줄 세우면
가장 위험한 설비가 맨 위로

02 · 짝이 되는 도구

필터링과 정렬은
한 짝

이상 설비를 찾는 것이 필터링,
무엇부터 손댈지 정하는 것이 정렬

정렬의 가치

집중의 한계와 흐름 파악 — 두 가지 가치

01 · 집중의 한계

100대를 동시 점검 불가
시급한 것부터 순서대로

사람이 한 번에 집중 가능한
대상이 제한적이기 때문

02 · 흐름 파악

시간순·값순 정렬로
숨은 패턴 발견

시간순으로 정렬하면 품질등급 변화 흐름이,
값순으로 정렬하면 극단적인 경우가 보임

sort_values 단일 키 정렬

sort_values 괄호에 기준 열 이름을 적으면 그 열로 정렬

sort_values 괄호에 **기준 열 이름**을 적으면 그 열로 정렬

기본은 작은 값부터인 **오름차순**

한 열로 정렬해도 **모든 열이 행 단위로 함께 이동**

```
df.sort_values('비스킷두께')
```

비스킷두께 열로 줄 세움 — 작은 값이 위로

sort_values 특징

행 통째 이동과 원본 보존 — 안전한 정렬

01 · 행 통째 이동

비스킷두께로 정렬해도
품질등급·사이클타임 함께 이동

엑셀에서 한 열만 정렬하다
데이터가 어긋나는 사고가 없음

02 · 원본 보존

정렬 결과는 새로 반환
원래 df는 그대로

마음 놓고 여러 기준으로
실험해도 원본은 안전

ascending과 다중 키 정렬

내림차순과 우선순위가 있는 다중 정렬

`ascending=False`를 주면 큰 값부터인 **내림차순**으로 정렬

기준 열을 **리스트로 묶으면** 우선순위가 있는 다중 정렬

```
df.sort_values('사이클타임', ascending=False)
```

사이클타임 큰 값부터 — 위험 우선 보기

```
df.sort_values(['품질등급', '사이클타임'])
```

먼저 품질등급으로, 품질등급 같으면 사이클타임로 — 우선순위 정렬

다중 키 정렬의 순서

리스트에 적은 순서가 곧 우선순위 — 방향도 열마다 별도

01 · 순서가 우선순위

리스트에 적은 순서가
1순위, 2순위

품질등급으로 큰 덩어리를 나누고
그 안에서만 사이클타임로 세부 정렬

02 · 방향도 별도

`ascending=[True, False]`

열마다 방향 지정

첫 기준은 오름차순,
둘째는 내림차순으로 자유 설정

정렬 후 상위 N개 추출

정렬과 상위 추출을 점으로 잇는 메서드 체이닝

정렬 후 **head**를 붙이면 상위 몇 개만 추출

정렬과 상위 추출을 **점으로 이어 한 줄**로 작성 가능

이렇게 동작을 점으로 잇는 것을 **메서드 체이닝**이라 함

```
df.sort_values('비스킷두께', ascending=False).head(5)
```


상위 N개 활용

메서드 체이닝과 실무에서의 활용

01 · 메서드 체이닝

`sort_values`에 바로
`.head`를 점으로 연결

왼쪽에서 오른쪽으로 읽으면
흐름이 자연스럽게 이어짐

02 · 실무 활용

점검 시급 상위 10대
중요한 일부만 추출

비스킷두께 가장 높았던 5건처럼
`head(숫자)`가 뽑을 개수

실습 5. 위험 순으로 정렬하기

목표

데이터를 위험한 순서로 정렬하고 상위만 추출

단계

- `sort_values`로 비스킷두께를 큰 값부터 내림차순 정렬
- `head`로 상위 다섯 개만 추출해 값 확인
- 여러 열을 리스트로 묶어 우선순위 다중 정렬

예상 결과

상위 5개 비스킷두께 값과 다중 정렬 첫 행 품질등급 출력

실습 6. 필터링과 정렬 연결

목표

조건으로 거른 결과에 정렬을 이어 붙이기

단계

- 고장 여부 조건으로 고장 설비만 먼저 거르기
- 거른 결과에 `sort_values`를 점으로 이어 비스킷두께 내림차순 정렬
- `head`로 상위 다섯 개만 남겨 샷 확인

예상 결과

5개 행, 비스킷두께 큰 순 샷 목록 출력

필터링 결과 저장과 copy

걸러낸 결과를 따로 저장해 수정하려면 `.copy()`로 독립 복사본

걸러낸 결과를 따로 저장해 수정하려면 **copy**로 복사본 생성

`df[조건].copy()`는 원본과 분리된 독립 복사본

복사본은 새 열 추가나 값 변경을 **경고 없이** 수행

위험 설비만 골라낸 뒤 **위험등급 열을 추가**하는 식의 작업에 적합

copy가 필요한 이유

원본인지 복사본인지 모호한 상황을 명확하게 분리

01 · 왜 필요

원본을 가리키는지
복사본인지 모호

원본을 가리키는 거면
메모가 원본에까지 남음

02 · 규칙

수정할 거면 `.copy()`
보기만 하면 불필요

복사기로 복사한 것과 같음
원본은 그대로 보존

SettingWithCopyWarning 이해

원본인지 복사본인지 모호할 때 뜨는 경고

원본인지 복사본인지 **모호할 때** 뜨는 경고

이름은 길지만 뜻은 단순 — 결과를 **장담 못 한다**는 경고

오류가 아니라 **경고**이므로 copy나 loc으로 해결

빨간 글씨로 길게 나와 당황하기 쉽지만 **코드는 멈추지 않고 끝까지 실행**

경고의 원인과 해결

.copy() 또는 .loc — 두 가지 해결 방법

01 · 원인

원본 일부인지
새 복사본인지 불확실

값을 바꾸면 원본까지 바뀔 수도,
복사본만 바뀔 수도 있어 예측 어려움

02 · 해결

따로 수정은 **.copy()**
원본 수정은 **.loc**

이 행의 이 열을 바꿔라라고
loc으로 명확히 지정

선택·필터·정렬을 잇는 사고 흐름

도구 하나하나보다 그것을 엮는 흐름이 분석의 진짜 힘

실제 분석은 선택, 필터링, 정렬을 **목적에 맞는 순서**로 연결

복잡한 질문도 **작은 단계로 쪼개면** 도구 조합으로 해결

도구 하나하나보다 그것을 엮는 **흐름**이 분석의 진짜 힘

예: 가장 위험한 설비 5대의 ID와 마모 수치 — **필터** → **정렬** → **상위 5** → **열 선택**

사고 흐름의 순서

먼저 거르고 줄 세운 뒤 마지막에 열을 추림 — 효율적 순서

01 · 전형 흐름

필터링 → 정렬 →
상위 추출 → 열 선택

도구들이 자연스럽게
연결되는 흐름

02 · 순서 기준

먼저 거르고 줄 세운 뒤
마지막에 열 추림

수천 개를 다 정렬하지 않고
일부만 거른 뒤 정렬하는 게 효율적

선택→필터→정렬 워크플로우

불러오기.확인.필터링.정렬.선택 — 다섯 단계의 흐름

분석은 **불러오기.확인.필터링.정렬.선택** 다섯 단계로 흐름

한 문장으로 외우기 — **불러오고, 확인하고, 거르고, 줄 세우고, 골라낸다**

1. 데이터를 불러오고
2. head·info로 구조 확인
3. 조건 필터링
4. 정렬로 우선순위
5. 열 선택·상위 추출

분석 워크플로우 5단계

불러오기 → 확인 → 필터링 → 정렬 → 선택

STEP 1

불러오기

CSV 등 데이터를 메모리로 가져오기. 분석의 시작

`read_csv`

STEP 2

확인

`head·info·columns`로 구조와 자료형을 점검

`head · info`

STEP 3

필터링

조건을 걸어 관심 행만 거름. `df[조건]` 패턴

`df[조건]`

STEP 4

정렬

`sort_values`로 위험 순으로 줄 세움

`sort_values`

STEP 5

선택

필요한 열만 추리거나 `head`로 상위만. 마침표

`[열] · head`

각 단계마다 쓰는 도구가 정해져 있어 단계만 떠올리면 코드도 자연스럽게 떠오름

각 단계의 도구

확인 단계와 처리 단계 — 모든 분석의 골격

01 · 확인 단계

`head · info · columns`

head로 생김새, info로 자료형,
columns로 열 이름 점검

02 · 처리 단계

`df[조건] · sort_values`

df[조건]으로 거르고,
sort_values로 우선순위 정렬

실습 7. 이상 의심 설비 리포트

목표

불러오기부터 판단 문장까지 전체 워크플로우를 두 데이터에 적용

단계

- 복합 조건으로 위험 설비를 거르고 비스킷두께 내림차순 정렬
- 필요한 주요 열만 선택하고 가장 위험한 설비로 판단 문장 작성
- 같은 흐름을 주소 로그 불량 데이터에도 적용해 결과 비교

예상 결과

주소 로그 위험 50건.판단 문장, 주소 로그 불량 상위 목록 출력

실습 — 마무리 정리

이 기본기 위에 앞으로의 모든 분석이 쌓임

두 데이터는 내용이 다른데 **왜 같은 방법으로 분석**되는가

행과 열을 고르고, **조건으로 거르고, 정렬하는 기본기**는 동일

이 기본기 위에 앞으로의 **모든 분석**이 쌓임

Pandas 데이터 선택과 필터링 파트 종료 — 다음은 결측치·이상치 전처리